

A TWO-PART DEEP DIVE · OPENAI CODEX CLI

The Codex Harness

Understand it, extend it, keep control of it

PART ONE · 01-05

Understand the runtime

harness · sandbox · config · AGENTS.md · context

PART TWO · 06-09

Extend and control it

MCP · skills · subagents · exec policy · hooks

Every claim here was checked against the Codex source, not just the documentation.

Three questions, and most people guess wrong

“Why did it run that without asking me?”

Part 2

Sandbox and approval are two separate settings.

“I put it in the config file — why is nothing different?”

Part 3

Six files can hold that setting — and the one you edited may not be the one that wins.

“It ignored my AGENTS.md.”

Part 4

It may never have been loaded at all.

None of these is the model being unpredictable. Every one is the harness doing exactly what it was told.

AGENDA

Nine parts, in two halves

UNDERSTAND THE RUNTIME

01

The harness

What sits between you and the model, and what it decides.

02

Sandbox vs. approval

Two different questions. Confusing them is the number one surprise.

03

Configuration

Six layers. Which one wins, and why the repo may not load at all.

04

AGENTS.md

How instructions are collected — and how they go missing silently.

05

Context & compaction

Why a long session gets expensive, and what to do about it.

EXTEND AND CONTROL IT

06

MCP

Giving Codex tools it does not have — and what that costs on every call.

07

Skills vs. subagents

Three things people keep mixing up. What loads when, and who decides.

08

Exec policy & hooks

Controls that run outside the model — and the one thing about hooks you must not forget.

09

Putting it together

One repo, all the pieces, plus what only shows up after real use.

01

The harness

What sits between you and the model

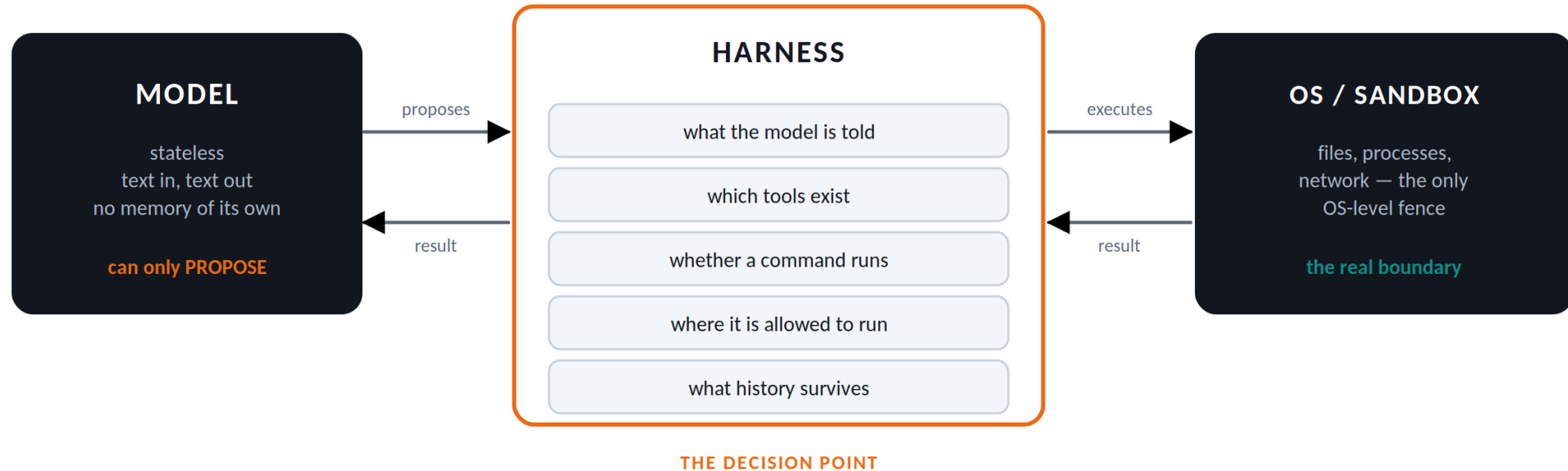
Model vs. harness

One message, many calls

What gets sent every time

Where the real boundary is

Codex is not “an LLM with a terminal”

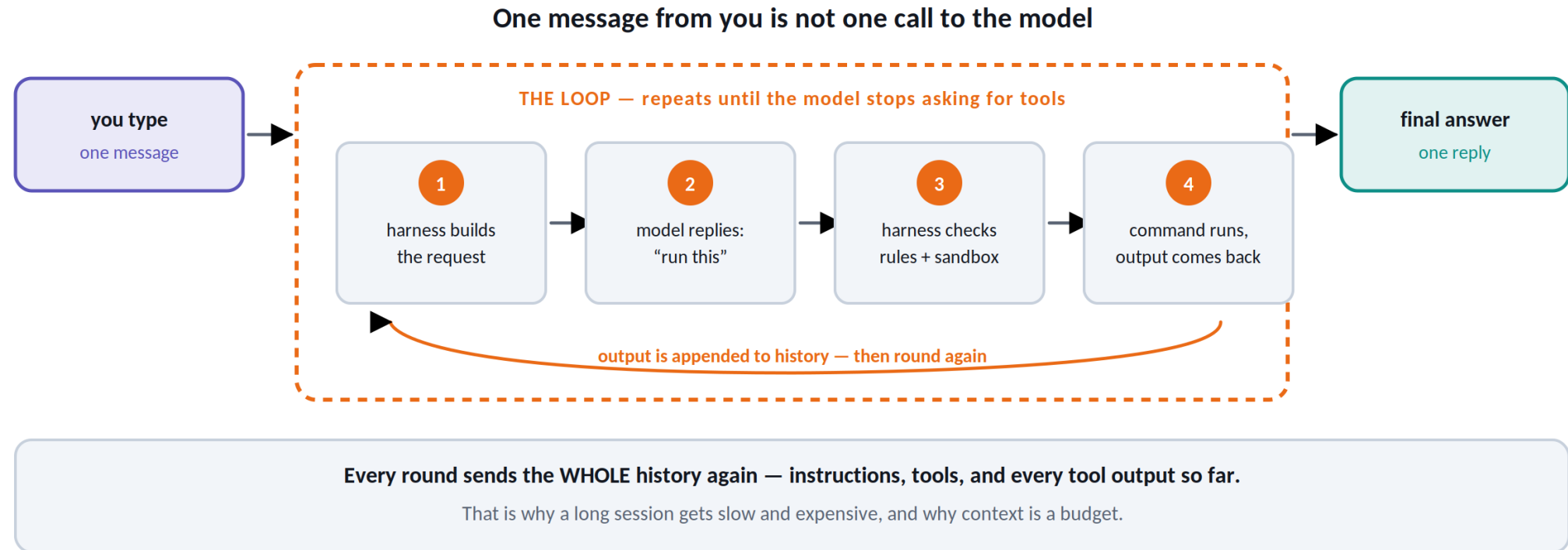


“The model proposes. The harness decides.”

Everything in both sessions is a way of steering that decision.

The model never runs anything. It writes a request. Something else decides what happens next.

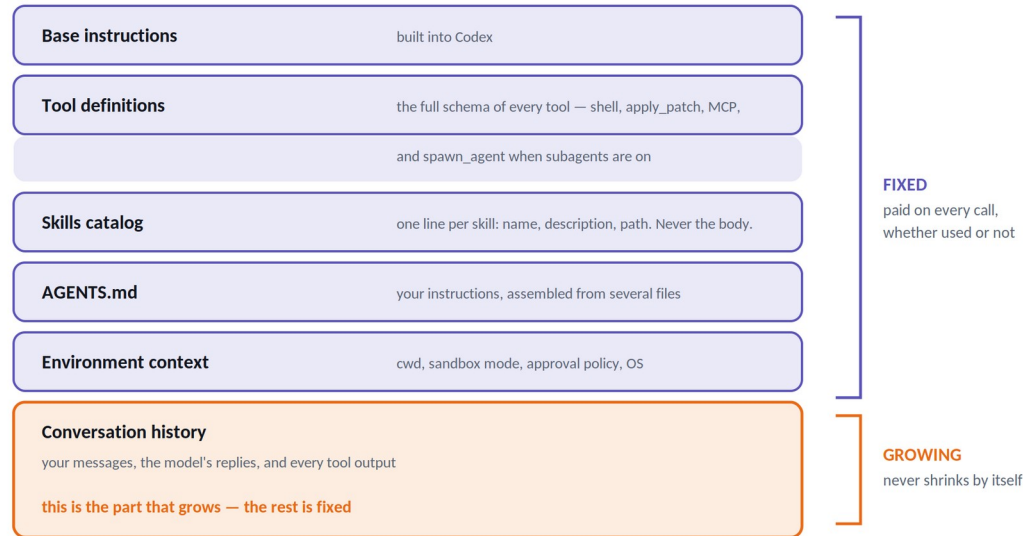
One message from you is many calls to the model



So “one question” can cost fifteen model calls — each one carrying everything that came before it.

What the harness sends, every single call

What the harness sends, every single call



Skills — the body comes later

- At session start the model gets a catalog only: one line per skill, capped at about 2% of the context window.
- The SKILL.md body arrives when the skill is actually used, not before.

Subagents — they ride in the tools

- No separate block. spawn_agent simply appears in the tool list when delegation is on.
- The list of roles sits inside that tool's own description — and only if you defined roles yourself.

How to see it for yourself

- **\$ codex debug prompt-input**
- Prints the exact message list the next turn would send, and makes no model call. One caveat: it prints the messages, not the tool schemas — so the real fixed cost is higher than what you see.

Everything above the orange block is paid on every call — even the skills you never use.

02

Sandbox vs. Approval

Two different questions people keep merging into one

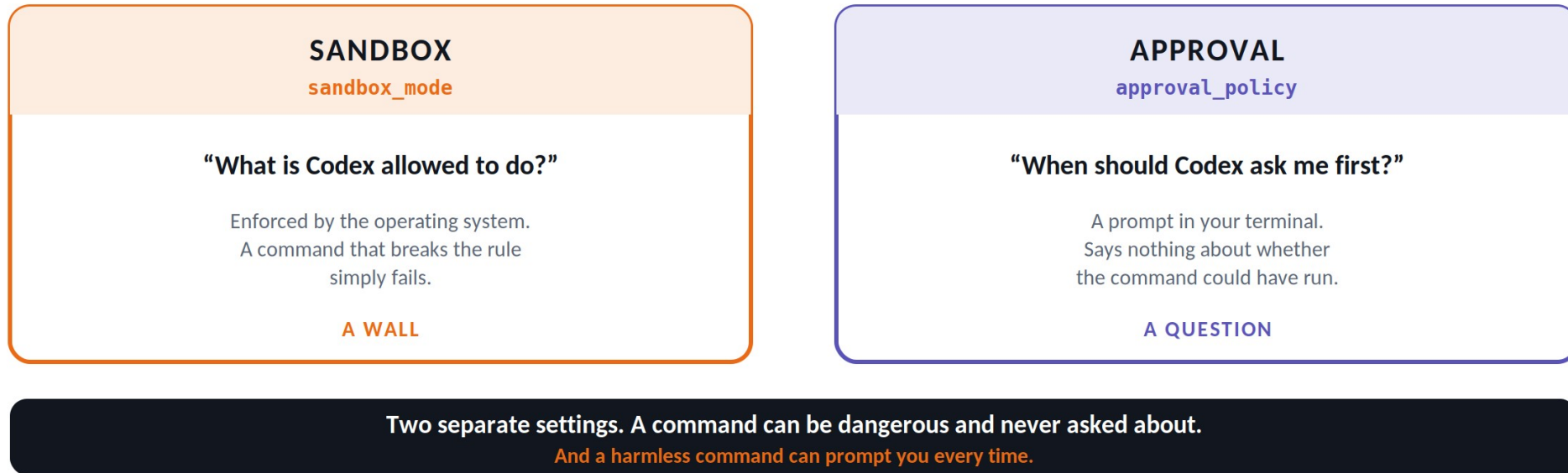
What it CAN do

When it ASKS you

The four combinations

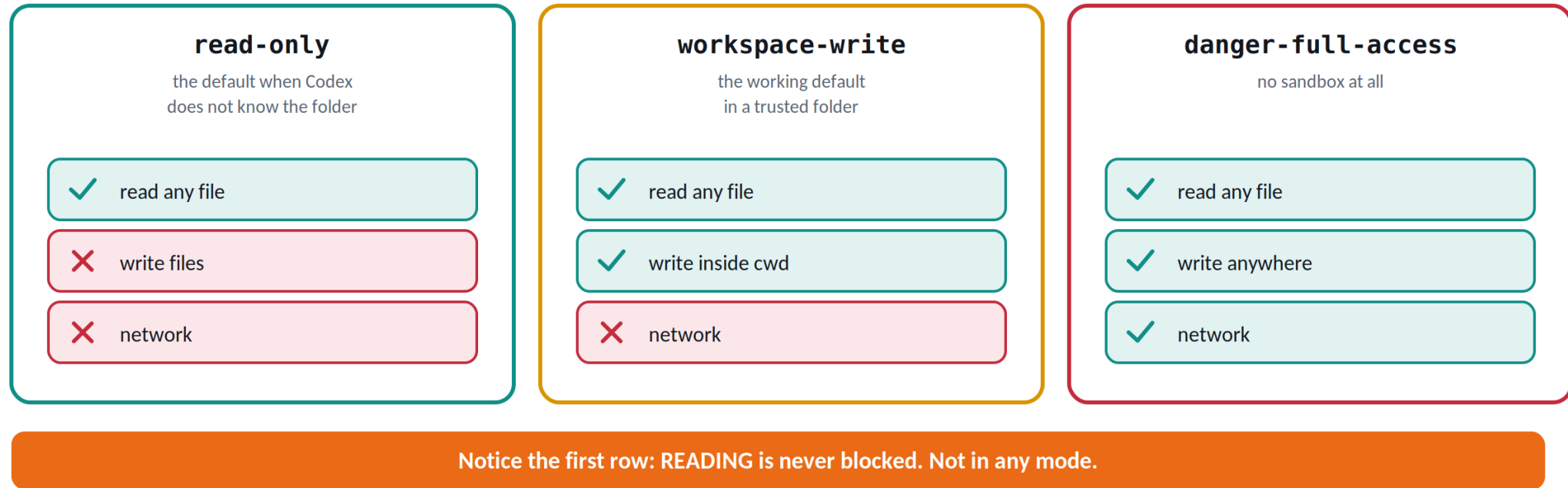
The gap nobody expects

Two settings, two completely different questions



- The mistake: treating an approval prompt as proof that something was risky, and silence as proof that it was safe.
- Neither is true. They are answering different questions.

Three modes — and what each one actually allows



Which one you get by default

- A folder you have marked trusted, or an ordinary project: workspace-write.
- A folder Codex has never seen: read-only — which is why it sometimes feels stricter than you remember.

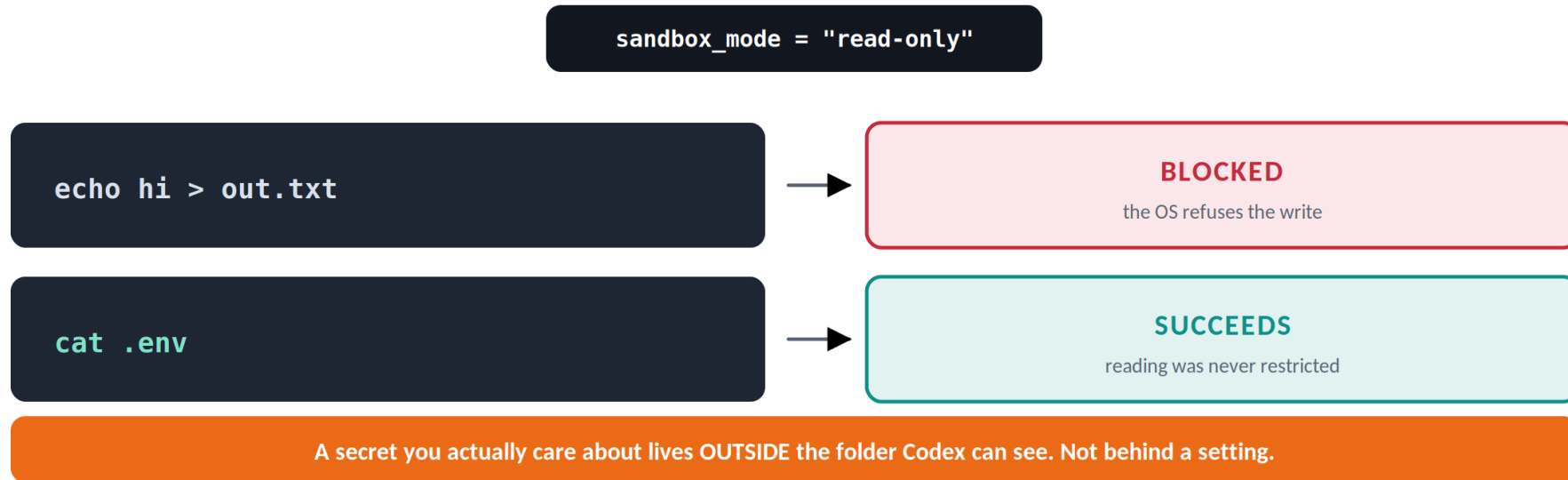
The same command, four different outcomes

The same command, four combinations			
approval ↓	read-only	workspace-write	danger-full-access
on-request	runs, sandboxed writing is refused by the OS	runs, sandboxed writes inside the repo succeed	runs, no sandbox and it never asked
never	runs, sandboxed no prompt — the sandbox is the answer	runs, sandboxed no prompt, writes succeed	runs, no sandbox nothing checks anything at all

“It asked me” is not a safety guarantee, and “it did not ask” is not a problem.

Under on-request in a sandbox, Codex stays quiet on purpose — the sandbox is already answering.

The gap: read-only does not mean it cannot read



What to do about it

- Do not keep production secrets in a folder Codex works in. That is the whole fix.
- A hook can block a read pattern, and Part 08 shows how — but a hook is a check, not a wall.

The sandbox controls what Codex can CHANGE. It was never a confidentiality boundary.

Where you set it, and how to check it

SET IT

```
# ~/.codex/config.toml  - permanent
sandbox_mode    = "workspace-write"
approval_policy = "on-request"

$ codex --sandbox read-only      # just this run
$ codex -c sandbox_mode='"read-only'"
```

CHECK IT, INSIDE A SESSION

```
> /permissions  change it for this session
> /status       what is in effect right now
> /debug-config  which config layer won, and why
```

The habit worth building today

- Never conclude “it is configured” because a file on disk says so. Conclude it from /status in the session you are actually in.
- Two minutes of this saves an afternoon of arguing with a setting that was never loaded.

Part 3 is about why the file on disk and the setting in effect are so often different.

03

Configuration

Six places hold settings. Only one of them wins.

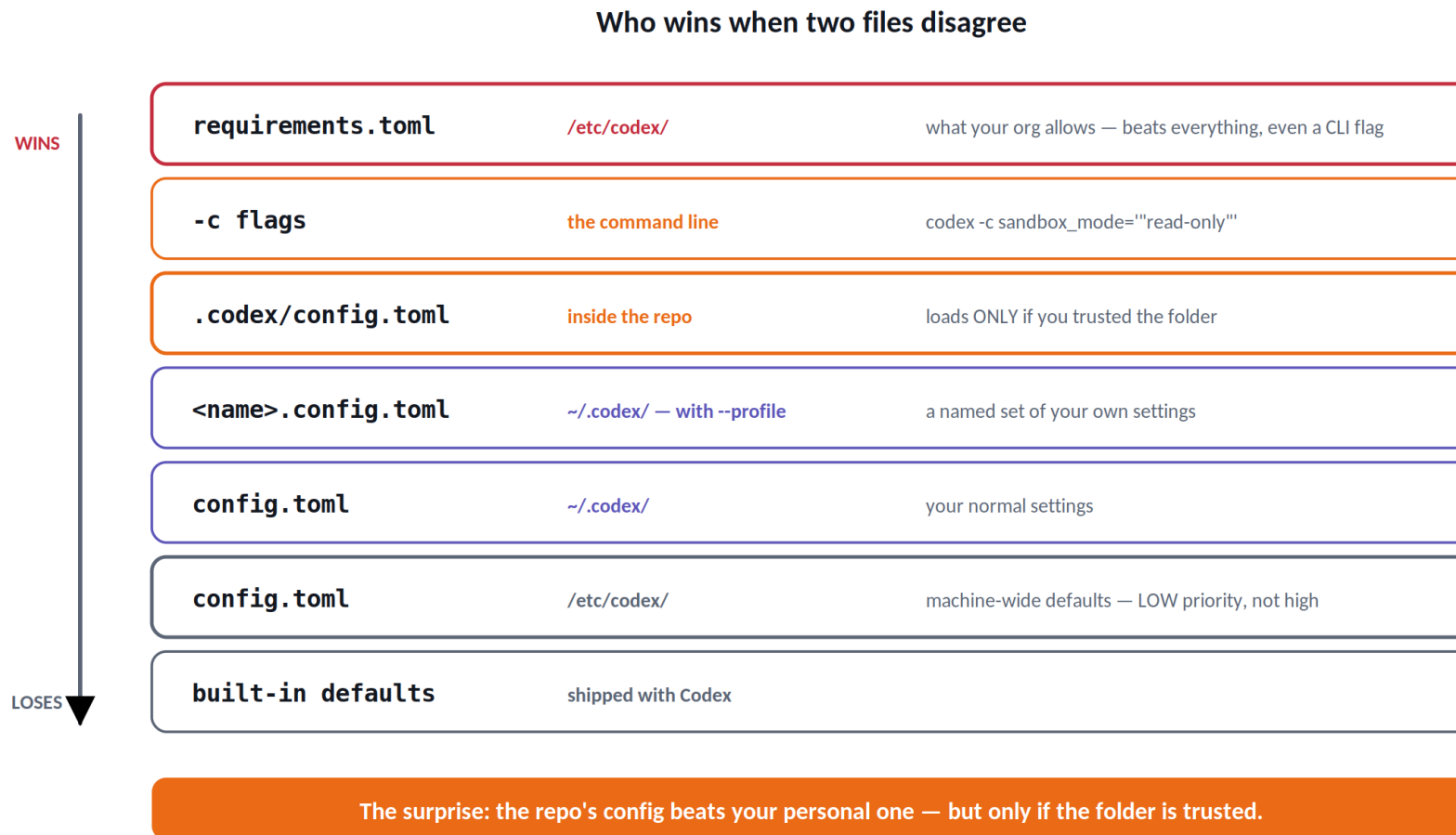
The precedence ladder

The repo can beat you

Trust decides what loads

How to see the winner

Who wins when two files disagree



vs CLAUDE CODE Five layers, same shape: managed > --settings > settings.local.json > settings.json > ~/.claude.

Why Codex asks before it trusts a folder

THE FIRST TIME YOU RUN CODEX IN A FOLDER

> You are in `~/work/some-cloned-repo`

Trust this folder? Codex can read, edit, and run files here, subject to your permission settings.

Folder settings can run code automatically, even without a model request.

Continue only if you trust these files.

1. Trust and continue 2. Quit

What it is protecting you from

- A repo you just cloned can ship its own `.codex/config.toml` — including a wider sandbox mode.
- It can ship hooks too, and a hook runs on its own, without the model asking for anything.
- **Without this question, git clone would be enough.**

NEEDS AN EXPLICIT YES

`.codex/config.toml`

hooks in the repo

exec policy rules in the repo

A folder Codex has never seen counts as NO.

LOADS ANYWAY

`AGENTS.md` in the repo

Only marking the folder untrusted stops this one.

Two different doors: the repo's SETTINGS stay out, but its INSTRUCTIONS still reach the model.

So a repo's settings “not working” usually means you never said yes. `/status` shows what is really in effect.

04

AGENTS.md

How your instructions reach the model — and how they go missing

Collected, not chosen

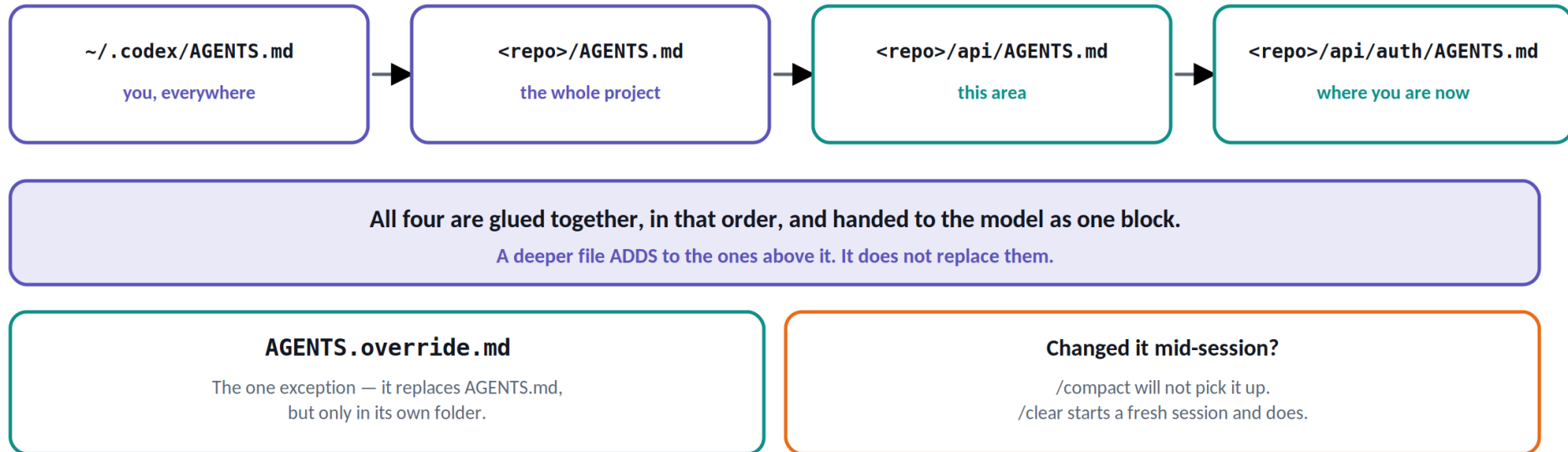
One shared budget

Silent truncation

Advice, not enforcement

Instructions are collected on the way down

Instructions are collected on the way down — not chosen

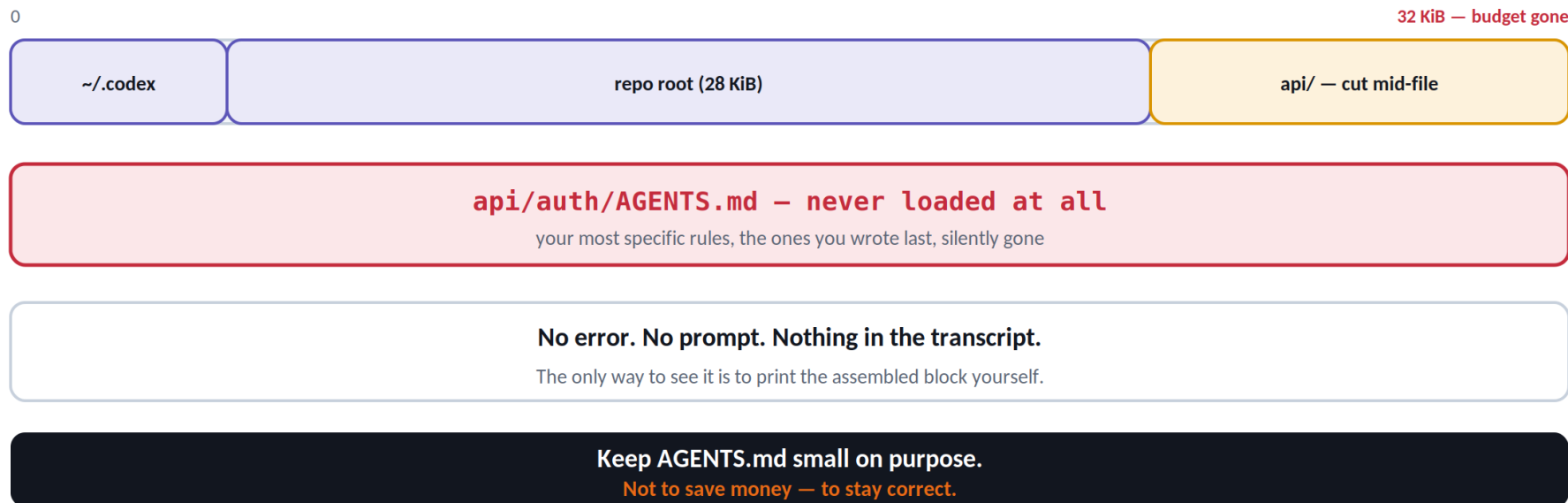


There is no “nearest file wins”. Every file on the path is glued together and sent as one block.

vs CLAUDE CODE CLAUDE.md concatenates the same way, and adds @imports plus .claude/rules/ with a paths: filter.

The 32 KiB budget — and the file that vanishes

One 32 KiB budget, spent from the top down



- The budget is spent from the top down. A large file at the repo root can use it all up before Codex reaches the one next to your code.
- The specific instructions you wrote most recently are the first to disappear — and nothing says so.

vs CLAUDE CODE No shared budget: 4 MiB per file, and a file over the limit is skipped rather than silently cut.

Instructions are advice. The sandbox is the boundary.

AGENTS.md says

- *“Never run migrations against production.”*
- The model reads it, and usually listens.
- Usually is not a guarantee — it is text competing with everything else in the context.

The sandbox says

- *“This process cannot write there.”*
- The operating system refuses. There is nothing to persuade.
- It holds even if the model never read a word of your file.

So the question to ask about any rule you write

- “Would I be upset if the model ignored this once?” If yes, it does not belong in AGENTS.md alone.
- Part 08 covers where it does belong: exec policy rules, hooks, and the sandbox settings behind them.

Write instructions to make the model useful. Write config to make the boundary real.

05

Context & Compaction

Why a long session gets slow, and what actually helps

A budget that only grows

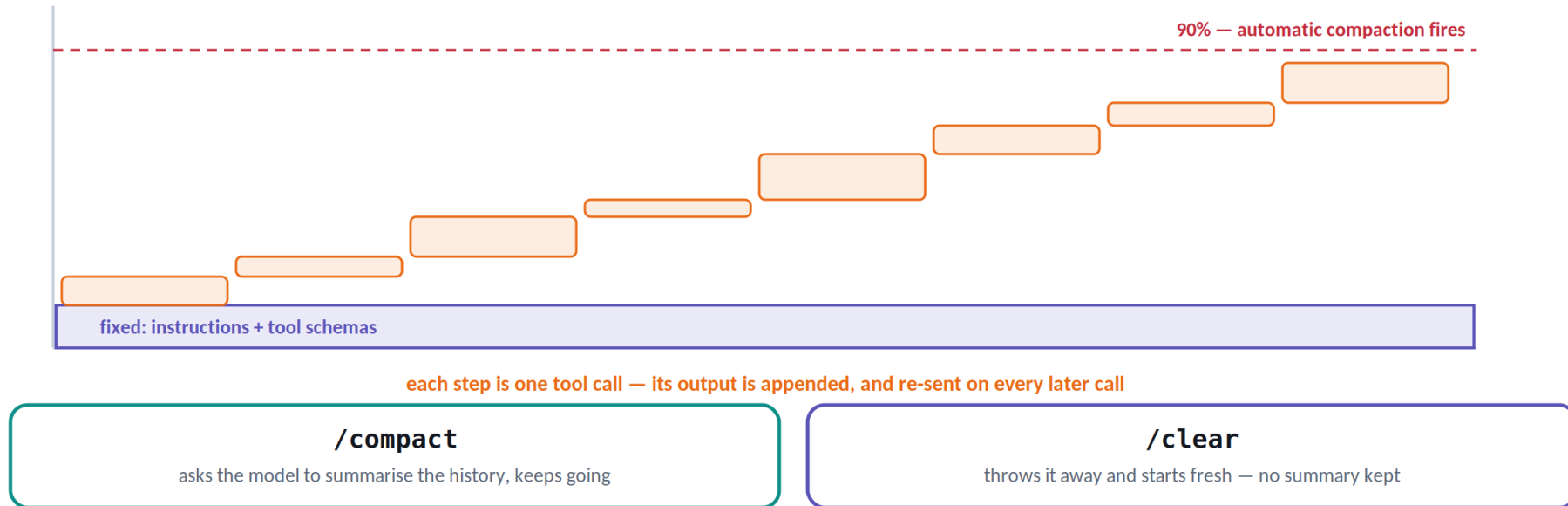
What compaction does

compact vs. clear vs. restart

When to reach for each

Context is a budget, and it only goes one way

Context only grows — until something removes it



- Automatic compaction fires at about 90% of the model's context window — and it can fire in the middle of a long task, between tool rounds.
- That is why long sessions sometimes seem to “forget” something halfway through. Nothing broke; history was summarised away.

/compact, /clear, restart — three different things

/compact	/clear	full restart
✓ keeps a summary	✗ keeps a summary	✗ keeps a summary
✓ same session	✗ same session	✗ same session
✗ re-reads config.toml	✓ re-reads config.toml	✓ re-reads config.toml
✗ re-reads AGENTS.md	✓ re-reads AGENTS.md	✓ re-reads AGENTS.md
✓ costs a model call	✗ costs a model call	✓ restarts MCP servers

Edited a config file? /compact will not see it. Reach for /clear, or restart.

The one that catches everybody

- You edit AGENTS.md or config.toml, stay in the same session, run /compact, and nothing changes — because /compact never re-reads them.
- /clear starts a fresh session and does pick them up. A full restart additionally re-reads machine-wide config and relaunches MCP servers.

Compact at a checkpoint you choose, not at ninety percent when the model chooses for you.

Five things to carry into the second half

The model proposes. The harness decides.

The model cannot run anything. Everything you configure is a way of steering that decision.

Sandbox and approval answer different questions.

“It asked me” is not proof of danger, and silence is not proof of safety.

Reading is never blocked, in any mode.

Secrets you care about belong outside the folder Codex works in.

The repo's config beats yours — if the folder is trusted. Otherwise it is not loaded at all.

Check with `/status` and `/debug-config`, never by reading a file.

AGENTS.md is one 32 KiB budget spent from the root down, and it is advice, not a boundary.

A big root file silently deletes your most specific rules.

Verify live. A file on disk is not a setting in effect.

END OF PART ONE

**You now know what the harness decides.
The rest is about steering it.**

06 · 07

Give it more reach

MCP servers, skills, subagents

08

Put real limits on it

exec policy, hooks, the sandbox

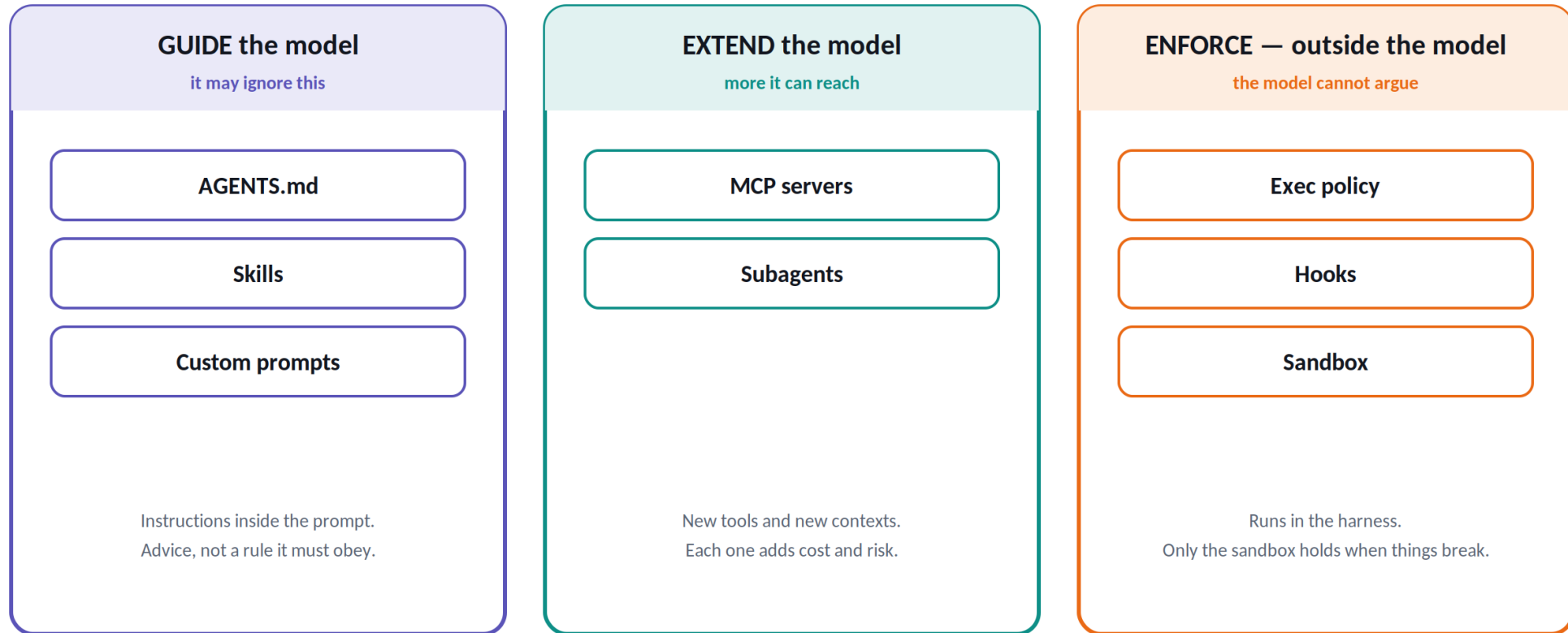
09

Make it hold together

one repo, and what experience adds

Everything from here on is optional. The five parts behind you are not.

Three jobs — and what handles each one



Pick the layer that matches the job — guiding, extending, or enforcing. Mixing them up is the number one mistake.

06

MCP

Extending what Codex can reach — and what that costs you

One flat tool list

Two ways to connect

What it costs every call

Where the sandbox stops

The problem it solves

Without MCP

- Codex can only reach what a shell command can reach from your repo.
- Tickets, error dashboards, design docs, staging data — all outside the repo, so you paste them in by hand.
- Every connection is custom code, written again for each tool and each agent.

With MCP

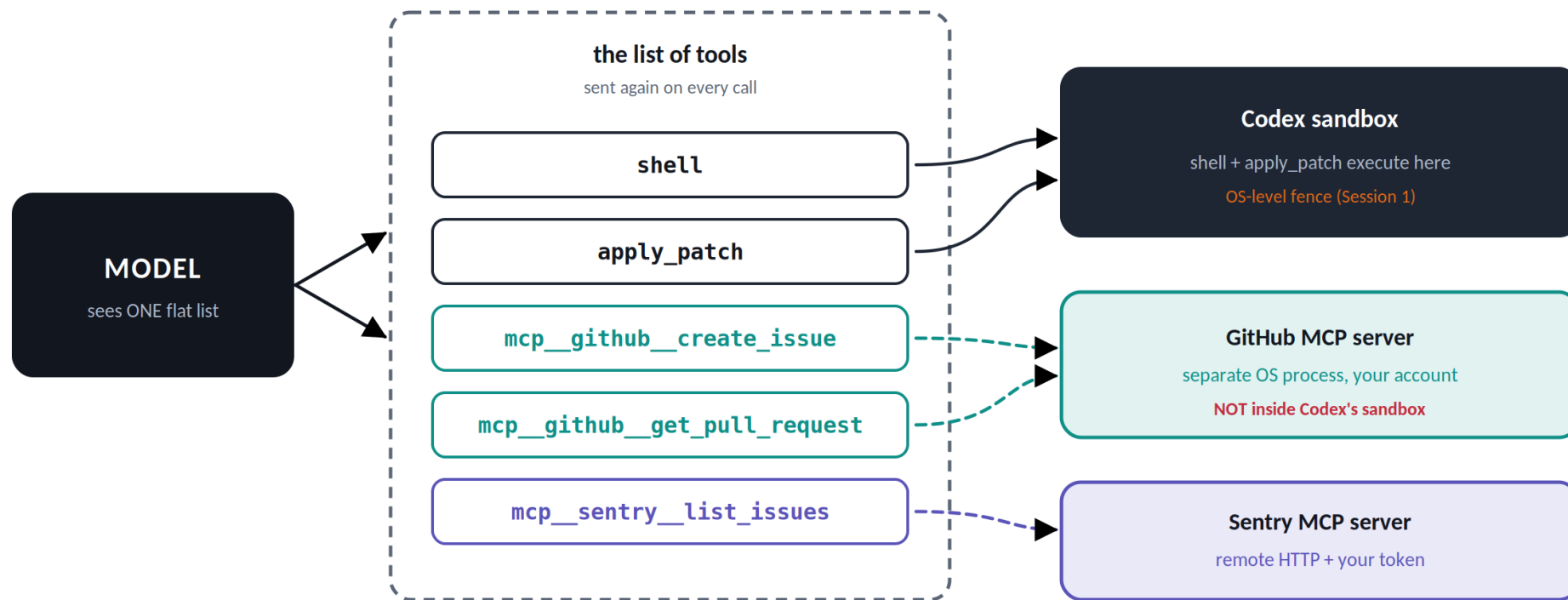
- One shared protocol. Codex is the client. Add a server and its tools join Codex's tool list.
- The same server works for other tools too — written once, not once per agent.
- Official servers exist for GitHub, Playwright, Jira/Confluence, Slack, Sentry, Stripe, Supabase.

The one fact everything else follows from

An MCP server's tools go into the SAME list of tools that already holds shell and apply_patch. There is no separate MCP channel, no separate sandbox, no separate permission system. To the model, calling `mcp__github__create_issue` and calling `shell` look identical.

Codex is an MCP client. Adding a server is a decision about trust, not about convenience.

One flat list — that is the whole design



To the model, calling an MCP tool looks exactly like calling shell. It cannot see the difference — and neither can you, unless you go looking.

The model sees no difference at all. Everything else in this section comes from that one design choice.

Two ways to connect — the whole difference, in the config

`stdio` — A PROGRAM ON YOUR MACHINE

```
[mcp_servers.github]
command = "npx"
args    = ["-y", "@modelcontextprotocol/server-github"]
env_vars = ["GITHUB_PERSONAL_ACCESS_TOKEN"]
```

*# Codex starts this as a subprocess on YOUR machine,
under YOUR account, with YOUR access.*

`streamable HTTP` — SOMEONE ELSE'S SERVER

```
[mcp_servers.sentry]
url          = "https://mcp.sentry.dev/sse"
bearer_token_env_var = "SENTRY_AUTH_TOKEN"
```

*# Someone else runs this. Your token travels to them,
and their code decides what to do with it.*

Where it runs

A program Codex starts, on your machine

Someone else's server, at a URL

How it authenticates

Environment variables passed to that program

`bearer_token_env_var`, or codex mcp login for OAuth

What you are trusting

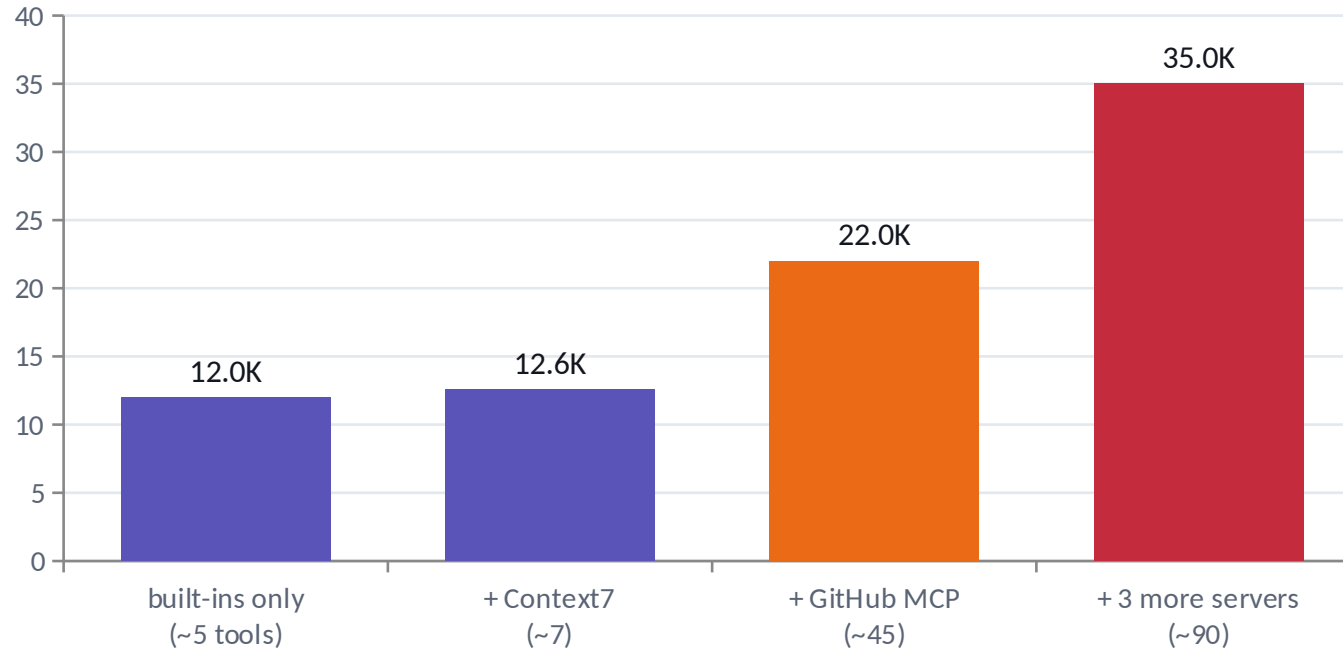
That npm package, running as you

That vendor, with that token

Neither one sits inside Codex's sandbox. And the file being on disk is no proof it loaded — `codex mcp list` and `/mcp` are.

What a server actually costs — every call, used or not

Illustrative — shape is reliable, exact numbers are not



Why it adds up

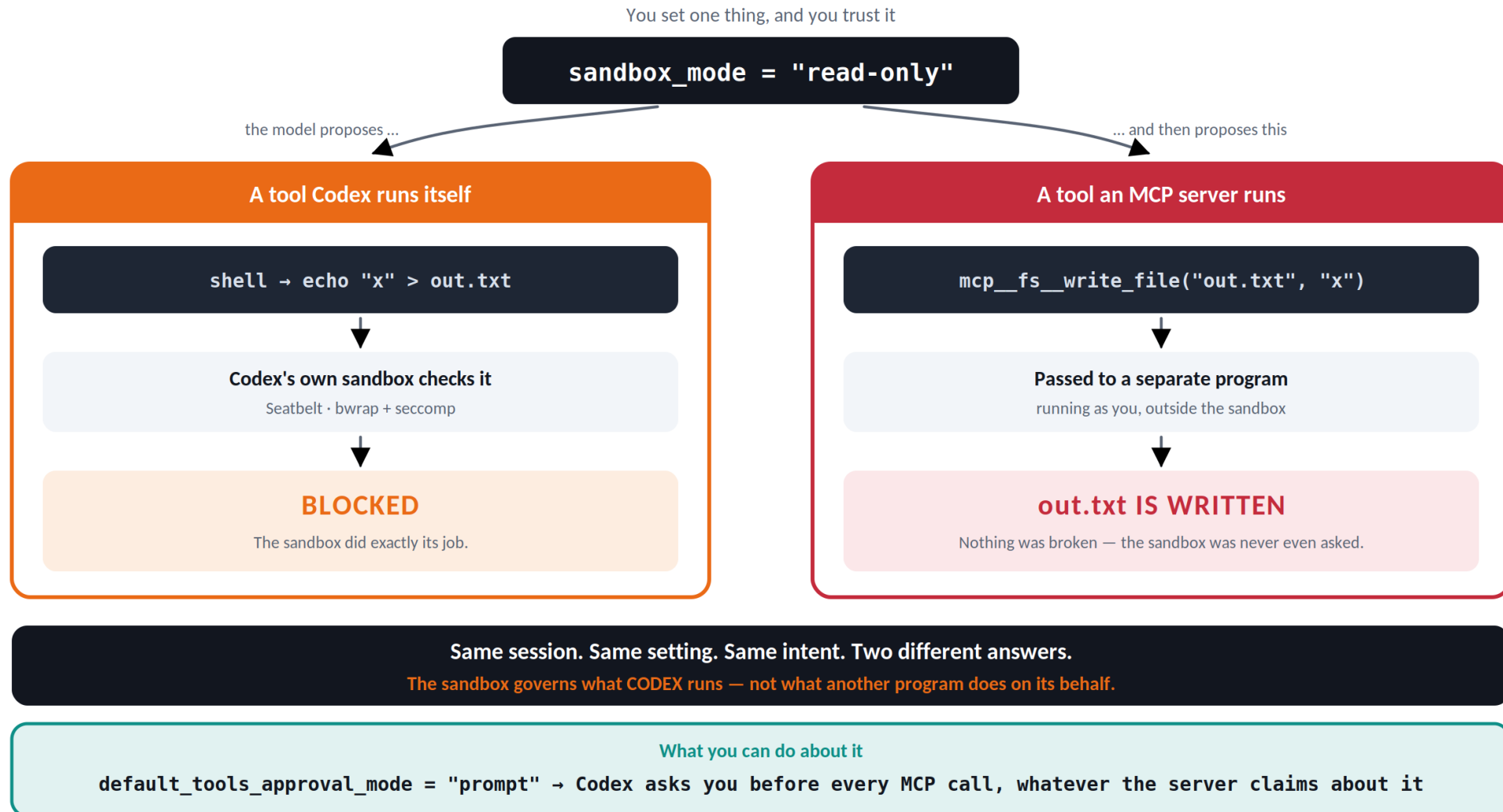
- The tool list sits at the top of the prompt — sent again on every model call, all session.
- Codex also includes the text a server sends at startup, so a wordy server costs extra.

What to do about it

- Add few. Three good servers beat nine hopeful ones.
- `enabled = false` instead of removing it — keeps the config, stops the cost.
- `enabled_tools` to keep only the tools you actually use.

A server you added "just in case" is a bill you pay on every single model call of every session.

The sandbox does not cover MCP — what that actually looks like



Takeaways

MCP tools merge into the same flat list as shell and apply_patch.

The model sees no difference — and there is no separate permission system.

Two transports: stdio (local subprocess, env-var auth) and Streamable HTTP (remote url, bearer or OAuth).

How it connects decides which trust question you should be asking.

Every server you add is a fixed cost on every model call, used or not.

Add few. Use `enabled = false` and `enabled_tools` instead of keeping ones you never use.

MCP config inside a project loads only if the project is trusted — otherwise it is skipped, silently.

Verify with `codex mcp list` and `/mcp`. Never with the file's presence.

MCP calls are NOT covered by Codex's OS sandbox or network proxy.

Whether you get asked depends on a label the server puts on its own tools; an open bug report shows that line being crossed with no prompt at all.

Trusting a server means trusting its code — not trusting Codex to hold it back.

07

Skills vs. Subagents

Three things people keep mixing up

What loads, and when

The description IS the trigger

A clean context, not more
speed

Who decides to run it

What a skill looks like — and where Codex looks for one

```
.agents/skills/pr-review/SKILL.md
---
name: pr-review
description: Review a pull request's diff for missing input
  validation on new Express routes, N+1 Prisma query patterns,
  and hard-coded secrets. Use when the user asks to review a PR,
  review a diff, or check code before merging.
---

## Instructions
1. Get the diff with `git diff main...HEAD`.
2. For each changed route handler, confirm the request body is
  validated before use.
3. Flag any Prisma query inside a loop as a likely N+1.
4. Search the diff for hard-coded keys or tokens.
5. Report findings ranked: blocking issues first.
```

Only two fields are required

- name and description. Nothing else is required at the top of the file.
- There is no trigger: field — writing one does nothing. The trigger words belong INSIDE the description.

Four places Codex looks

REPO \$CWD/.agents/skills, parents, repo root

USER \$HOME/.agents/skills

ADMIN /etc/codex/skills

SYSTEM bundled with Codex

Symlinked folders work — Codex follows the link. Useful for one shared skills repo, cloned once and linked in.

To call one by hand: /skills, or type \$ and its name.

Loaded only when needed — why 30 skills cost almost nothing

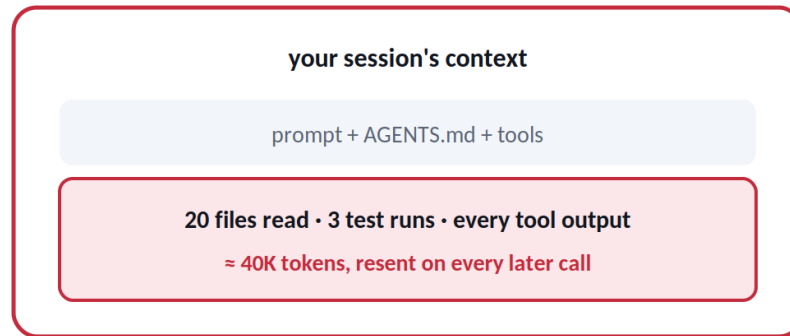


30 skills cost almost nothing until one actually matches. AGENTS.md and MCP tool lists have no way to do this.

The docs call this "progressive disclosure". MCP tool lists have no way to do it — a skill costs almost nothing until it is needed, an MCP server costs from the moment you add it.

Why they exist — and it is not speed

WITHOUT delegation



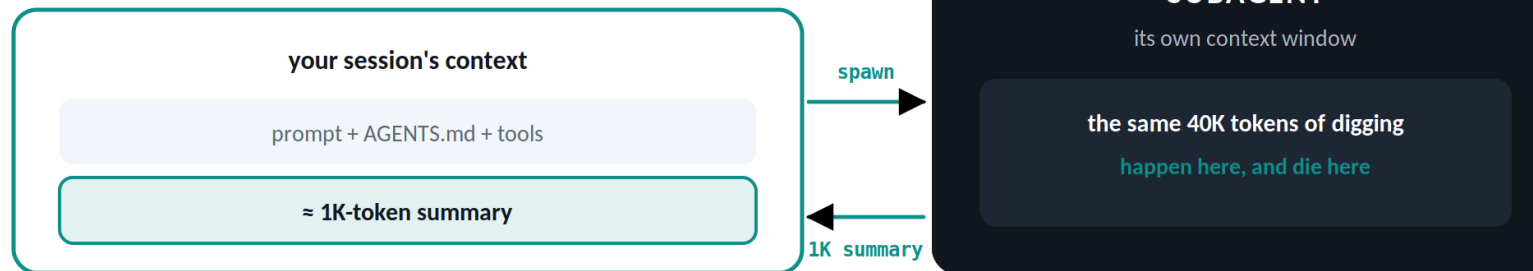
The trade you are actually making

Total tokens go UP — each subagent does its own model and tool work.
What goes DOWN is what lands in YOUR context and gets resent forever.

So: hand off digging and checking. Do not hand off for speed.

Official docs say this twice: subagent workflows consume more tokens.

WITH delegation



Compaction shrinks history that already arrived. Handing off stops it arriving at all.

Hand off digging and checking. Do not hand off because you want it to feel faster.

Handing off is just tools — and five ways it starts

THE MODEL SEES THESE AS NORMAL TOOLS

```
spawn_agent    send_input    resume_agent
wait           close_agent
```

- Because it is just tool calls, handing off goes through the same history and compaction as everything else.
- "Model proposes, harness decides" applies here too — spawning is a proposal.

How	Who decides	Best for
Ask in plain language	You — the model splits the work	One-off, most flexible
An AGENTS.md rule	Written once, then automatic	Things you repeat in this repo
A skill instructs it	Automatic when the skill fires	Steps you reuse
<code>codex --cd agents/xxx</code>	You, explicitly	Running one named agent
<code>codex exec --profile xxx</code>	You, via a profile	CI and scripts

The plain-language version people actually use

"Spawn three agents in parallel: one runs the test suite, one checks for missing docstrings, one reviews src/ for security issues. Summarize the results for me."

agents.enabled defaults to true — you turn handing off OFF, you do not turn it on. Older "codex features enable multi_agent" instructions are out of date.

An agent with a role — and what a role can actually change

A NAMED ROLE

```
# ~/.codex/agents/reviewer.toml

description = "Reviews a diff. Never writes code."

developer_instructions = """
You review, you do not fix. Never edit a file.
"""

model = "gpt-5.6"

[features]
shell_tool = false      # this reviewer gets no shell at all
```

How you actually narrow a role

- [features] can only turn things OFF, and only these five: shell_tool, apps, plugins, memories, request_permissions_tool.
- [skills] works the same way — a role can disable a skill, never enable one.
- shell_tool = false is the strong one: that agent cannot run a command at all.

It inherits yours — this matters

- "Subagents inherit your current sandbox policy." Documented, and confirmed in the source.
- That includes changes you made mid-session: --yolo, or opening up /permissions, carries to every subagent you start.
- The child starts from the parent's effective config, then a small set of role overrides is layered on top.

What a role file CANNOT change

- **sandbox_mode** · **approval_policy** · **mcp_servers**
- These parse without an error and are then ignored. The overrides a role can apply are a fixed list, and none of these is on it.
- So a subagent always runs with the parent's sandbox and the parent's MCP servers. You cannot hand one role its own server.

vs CLAUDE CODE Its subagent file really applies tools, mcpServers, permissionMode, model, and isolation: worktree.

A role file, field by field

EVERY FIELD A ROLE CAN ACTUALLY APPLY

```
# ~/.codex/agents/reviewer.toml

name = "reviewer"           # optional – filename is used if omitted

# REQUIRED. The model reads this to decide WHEN to pick this role.
description = "Reviews a diff for security. Never writes code."

# REQUIRED in a role file. Sent as a `developer` message –
# THIS is how you brief the subagent.
developer_instructions = """
You review, you do not fix.
Rank findings by severity. Never edit a file.
"""

nickname_candidates = ["Ada", "Hopper"]    # cosmetic

model = "gpt-5.6"
model_reasoning_effort = "high"            # low | medium | high

[features]                                # disable-only
shell_tool = false

[skills]                                  # disable-only, same idea
config = [{ name = "deploy", enabled = false }]
```

The two that decide behaviour

description

— how the model picks this role. Write it like a skill description.

developer_instructions

— the brief itself, sent as a developer message. Required: a role file without it is rejected at load.

The override list is fixed

- Nine slots, and that is all: `developer_instructions`, `model`, the three reasoning / verbosity settings, `personality`, `service_tier`, `features`, `skills`.

Check your own version

- `agents.max_depth` exists but the source marks it "ignored by V2", and `agents.job_max_runtime_seconds` now does nothing.

Anything else you put in this file — `sandbox_mode`, `mcp_servers`, `approval_policy` — parses cleanly and is then ignored.

The same request, four mechanisms, four outcomes

What you actually want: "make sure this PR is safe to merge"

Custom prompt

~/.codex/prompts/pr-check.md exists with the checklist.

Asking in plain language does NOTHING. Codex has no way to know the file exists unless you type /prompts:pr-check.

Skill

The pr-review skill exists with a well-written description.

The exact same plain-language question fires it. No slash command anywhere.

Subagent, you asked

"Spawn a subagent to check if this PR is safe to merge."

Sent to a separate context — but only because you said spawn. The plain question alone would have run it right here.

Subagent, standing rule

AGENTS.md: "Always delegate PR-readiness checks to a subagent."

The same plain question now also spawns. Looks identical to the skill from outside — different mechanism entirely.

Identical user input. Different outcome. The only variable is WHICH MECHANISM holds the instructions.

What would NOT happen locally: Codex deciding on its own to check a PR you never mentioned, with no request and no rule anywhere in the project.

Takeaways

A SKILL.md needs only name and description. There is no trigger field.

The description is the only thing Codex matches on — put the words a real person would type at the front.

Skills stay cheap: only name + description load until one matches, capped at about 2% of the context window.

AGENTS.md and MCP tool lists have no way to do this.

Subagents exist for context isolation, not speed.

40K of digging happens in a throwaway context and comes back as 1K. Total tokens go up; your context stays clean.

A subagent always runs with the parent's sandbox and the parent's MCP servers.

sandbox_mode and mcp_servers in a role file parse cleanly and are then ignored. To narrow a role, use [features] shell_tool = false.

Local Codex delegates only when you asked, or a standing rule told it to.

Never with no instruction anywhere. The limits on threads and depth do not always hold — there are known bugs.

Skills shape how it thinks. Subagents move where the thinking happens. Neither one is a real barrier.

08

Exec Policy & Hooks

Controls that run outside the model

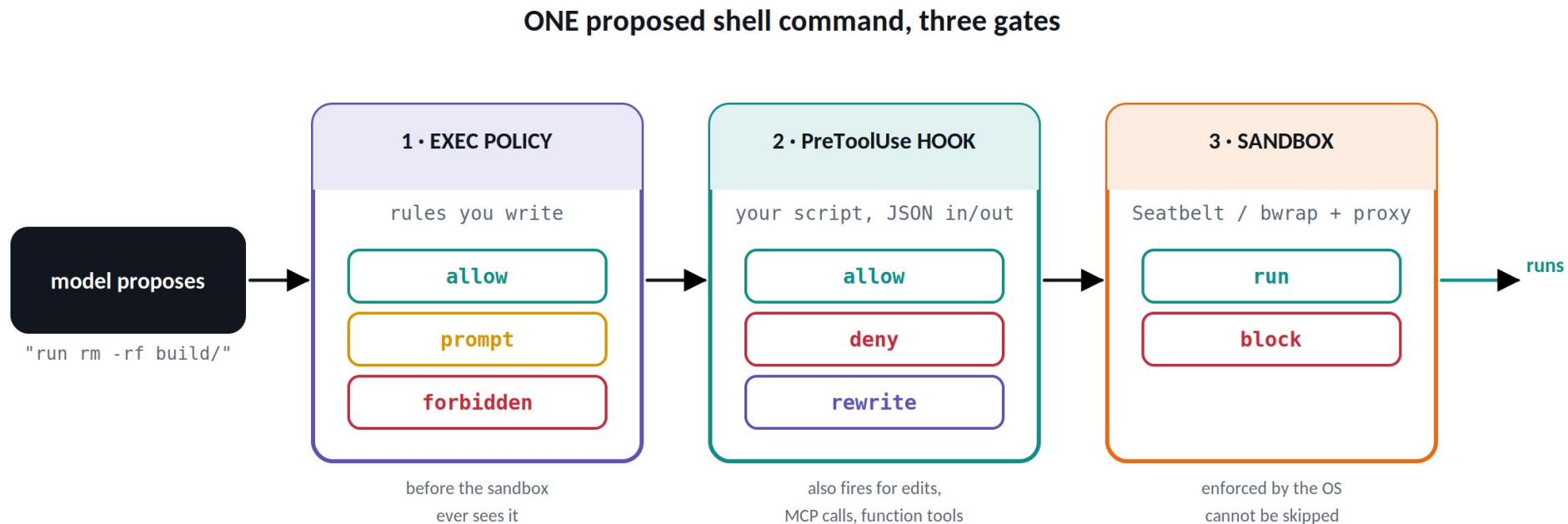
Three checks per command

Rules you write

12 points in a session

Hooks let things through
when they break

Three checks between "the model proposed it" and "it ran"



Three separate checks, in this order — a command must survive all three.

The first two are rules. Only the third is actually enforced.

A rule file that runs between "proposed" and "ran"

STARLARK — A SMALL, PREDICTABLE RULE LANGUAGE

```
# ~/.codex/rules/default.rules

prefix_rule(
    pattern = ["pytest"],
    decision = "allow",
    justification = "Running the test suite is always safe here",
)

prefix_rule(
    pattern = ["git", ["push", "reset"]],      # a list = any of these
    decision = "prompt",
    justification = "History-altering git operation",
    match      = ["git push origin main", "git reset --hard"],
    not_match  = ["git status", "git log"],
)

prefix_rule(
    pattern = ["rm", "-rf"],
    decision = "forbidden",
    justification = "Use git clean instead — recoverable",
)
```

How matching works

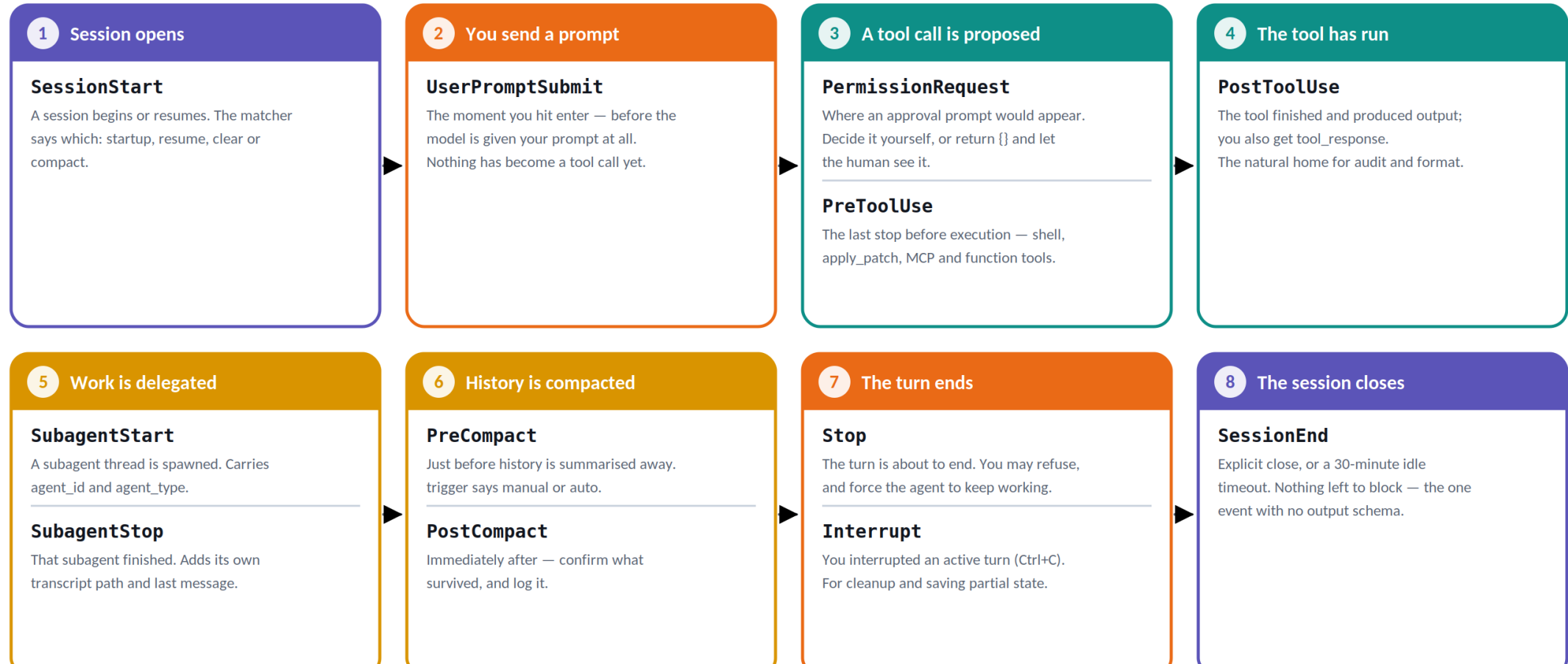
- Words match in order, from the start only — it checks how the command BEGINS, not what comes later.
- A list inside the list means "any of these here".
- decision defaults to allow if you omit it.
- The strictest one wins: forbidden > prompt > allow. A higher layer cannot loosen a stricter one.

match / not_match are not comments

- They are unit tests that run when the rule file loads.
- A rule whose own examples do not match fails at startup — you find the typo then, not when something goes wrong.

```
$ codex execpolicy check --pretty \
  --rules ~/.codex/rules/default.rules \
  -- git push origin main
```

Twelve events, and exactly when each one fires



Moments 3 and 4 repeat inside a single turn, for as long as the model keeps proposing tools. Hooks are ON by default: [features] hooks = false turns them off.

How a hook talks to Codex — JSON in, JSON or an exit code out

STDIN — COMMON FIELDS + EVENT-SPECIFIC ONES

```
{
  "session_id": "abc123",
  "transcript_path": "/path/to/transcript.jsonl",
  "cwd": "/path/to/project",
  "hook_event_name": "PreToolUse",
  "model": "...",
  "permission_mode": "default",
  "turn_id": "turn-42",
  "tool_name": "Bash",
  "tool_input": { "command": "cat .env" }
}
```

STDOUT — TO BLOCK A TOOL CALL

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Reading .env is blocked."
  }
}

# equivalent: exit 2, reason on stderr
# exit 0 + no output = fine, do nothing
```

transcript_path is on every event

- It points at the full session transcript Codex is already writing on disk.
- The event fields show one moment. That file has the whole conversation.

Two things to remember

type = "command" runs a script. type = "mcp_tool" calls an MCP tool.

Any exit code other than 0 — or output that is not valid JSON — counts as the hook itself failing. That matters in two slides' time.

PreToolUse can also rewrite the command

model proposed: `pytest`

→

what actually ran: `pytest --maxfail=1`

ALLOW IT — BUT SWAP THE COMMAND FIRST

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow",
    "updatedInput": { "command": "pytest --maxfail=1" }
  }
}
```

When this helps

- Always add a flag you want: `--maxfail`, `--dry-run`, a timeout.
- Turn a broad command into the narrow one you meant to allow.

A rewrite is invisible in the transcript. If you rewrite, log what you replaced — or nobody will understand that command later.

Example — stop the agent from reading .env

Why bother: a read-only sandbox blocks writing and network — but not reading. `cat .env` works in every sandbox mode.

1 • The model proposes

```
cat .env
```

A normal shell command. Nothing has run yet.

2 • Your script sees it first

```
if ".env" in cmd:
```

Codex sends the proposed command to your script, as JSON on stdin.

3 • Your script says deny

```
"permissionDecision": "deny"
```

Codex refuses the call and tells the model why. The command never runs.

```
# ~/.codex/hooks/block_env.py - the whole script
cmd = json.load(sys.stdin)["tool_input"]["command"]
if ".env" in cmd:
    print('{"hookSpecificOutput": {"hookEventName": "PreToolUse", "permissionDecision": "deny"}}')
```

```
# config.toml - two lines to wire it up
[[hooks.PreToolUse.hooks]]
type = "command"    command = "python3 ~/.codex/hooks/block_env.py"
```

The sandbox would have allowed that read. The hook is the only thing that stopped it.

The event most people never think to use

UserPromptSubmit — fires before the model reads your message

SOMEONE PASTES A TOKEN INTO CHAT

```
e      = json.load(sys.stdin)
prompt = e.get("prompt", "")

if SECRET_RE.search(prompt):  # sk-, ghp_, AKIA, PEM...
    print(json.dumps({"decision": "block",
                      "reason": "That looks like a live key."}))
    sys.exit(0)
```

PreToolUse cannot catch this

- It only fires on a proposed tool call. Pasted text never becomes one.
- By then the key is already at the model and already on disk.

The wrong event does not give you a weaker check — it gives you no check, and no error. Ask when the thing you care about first exists.

Three more events — and one script to handle them all

PostToolUse

- Runs after a tool finished.
- Format every file the agent edits, or write one audit line per call.

PreCompact

- Runs just before history gets summarised away.
- Your only chance to save the transcript before it is gone.

PermissionRequest

- Runs where an approval prompt would appear.
- Auto-approve the safe cases, leave the rest to a human.

ONE DISPATCHER, ONE LOG FILE

*# Do not write one script per event. Point every event at the SAME file,
then branch on hook_event_name inside it.*

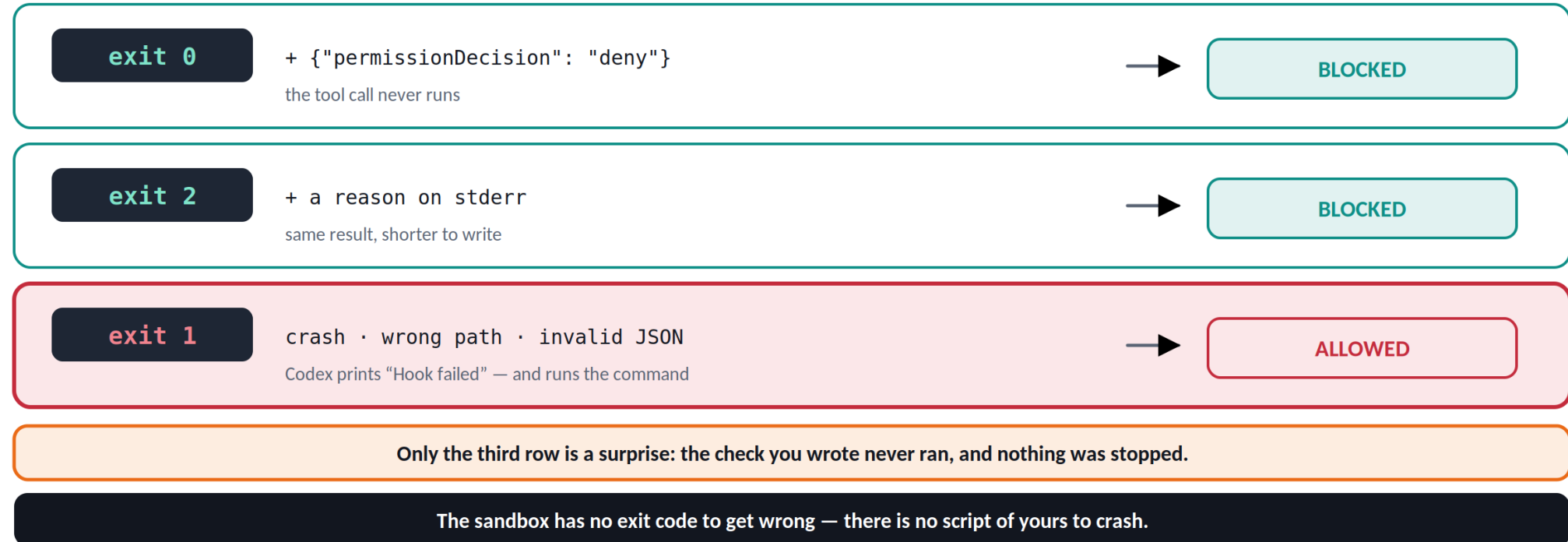
```
[[hooks.PreToolUse]]      [[hooks.PostToolUse]]      [[hooks.PreCompact]]  
type      = "command"  
command = "node .codex/hooks/dispatch.js"      # ...the same one, every time
```

```
$ tail -f .codex/hooks.log      # every event, one file, in order
```

Hooks are hard to debug because you cannot see which one ran. One script and one log file is the whole fix.

Hooks fail open — a broken hook lets the action through

What Codex does with your hook's exit code



The transcript says Hook failed. The command still ran. A message is not a barrier.
And under `codex exec --json` the hook events are not emitted at all, so CI sees neither the failure nor a block.

vs CLAUDE CODE Same contract, same fail-open: its docs say exit code 1 is non-blocking and the action proceeds.

Choosing the right mechanism

You want to...	Use	Why that one
Block or prompt on a specific command pattern	Exec policy – prefix_rule	Runs before the sandbox, structured allow/prompt/forbidden, and smart approvals will draft the rules for you.
A barrier a broken script cannot get around	sandbox_mode + permissions	Enforced by the operating system. It cannot break open. This is the only row where "guaranteed" is an honest word.
Log everything the agent does	Hook – PostToolUse	Fires after every supported tool call, for every tool kind, including MCP.
Save state before compaction discards it	Hook – PreCompact	The one moment designed for exactly this. Pair it with PostCompact to confirm.
Load project context at session start	Hook – SessionStart	Fires once, at the right moment, on startup / resume / clear / compact.
Auto-format after every edit	Hook – PostToolUse on apply_patch	Made exactly for reacting to one specific tool having just run.

Hooks are for watching and small automation. Exec policy and the sandbox are for protection. If it must still hold when your script crashes, it does not belong in a hook.

Takeaways

Three checks on every proposed command: exec policy → PreToolUse hook → sandbox.

The first two are rules. Only the third is actually enforced.

prefix_rule matches the start of a command only; the strictest decision wins; match / not_match are tests that run when the file loads.

Project rules live in <repo>/codex/rules/ and load only if the project is trusted — the fourth time today that trust decides whether config loads.

Hooks are ON by default and fire at 12 points in a session. PreToolUse can rewrite the command, not just allow or deny it.

Instructions telling you to enable `codex_hooks = true` are out of date.

Windows needs `command_windows` / `commandWindows` — the top cause of a hook silently not running.

Hooks you wrote yourself must be approved once, tracked by a hash of the file — and Codex needs a full restart after any change.

Hooks fail open — a broken script is reported, but the action still runs.

Only exit code 2 blocks. Treat a hook like a linter, not a firewall — the sandbox is the only layer that cannot break this way.

Never let a hook be the only barrier for something that matters.

09

Putting it together

One repo, all the mechanisms, and what experience adds

Guide, extend, or enforce

A real setup, end to end

Mistakes and tips from real
use

Onboard it like a new hire

Ask one question per requirement

One repo, all seven mechanisms, each doing only its own job

GUIDE — inside the prompt, the model may still ignore it

AGENTS.md

stack, commands, conventions · keep it small

Skills

one per recurring workflow · description is the trigger

agents/<role>/AGENTS.md

what each delegated role is for

EXTEND — more reach, more cost, more to trust

MCP: 1–3 servers

checked source · narrow token · enabled=false for the rest

reviewer.toml

a brief + shell_tool = false — the one real narrowing

explorer subagent

spends 40K digging, hands back 1K

ENFORCE — outside the model entirely

.codex/rules/*.rules

forbid what must never run · ask about what must be seen

hooks.json

audit log, secret scan, auto-format — observation

sandbox_mode

the real barrier · git commit is the real undo

Ask one question per requirement: am I guiding, extending, or enforcing?

Anything that MUST hold goes in the bottom band. Everything else is only advice.

What that looks like in one real repository

A SETUP YOU COULD DEFEND IN A REVIEW

```
taskflow-api/
├── AGENTS.md                # stack, commands. SMALL.
├── src/payments/AGENTS.md   # local rules, sensitive area
├── .agents/skills/
│   ├── pr-review/SKILL.md   # the recurring checklist
│   └── db-migration/SKILL.md # the 6-step migration dance
├── agents/
│   └── reviewer/AGENTS.md   # what this role is for
└── .codex/
    ├── config.toml          # sandbox_mode, approval_policy,
    │                        # mcp_servers, agents.*
    ├── rules/team.rules     # forbidden + prompt commands
    └── hooks/
        ├── hooks.json       # every event -> one dispatcher
        └── dispatch.js      # switch on hook_event_name

# plus ~/.codex/agents/reviewer.toml
# -> [features] shell_tool = false
```

Guide

- AGENTS.md stays short enough to survive the 32 KiB cap — detail moves into skills.
- A local AGENTS.md next to sensitive code, because the files are joined together from the repo root down to your folder.

Extend

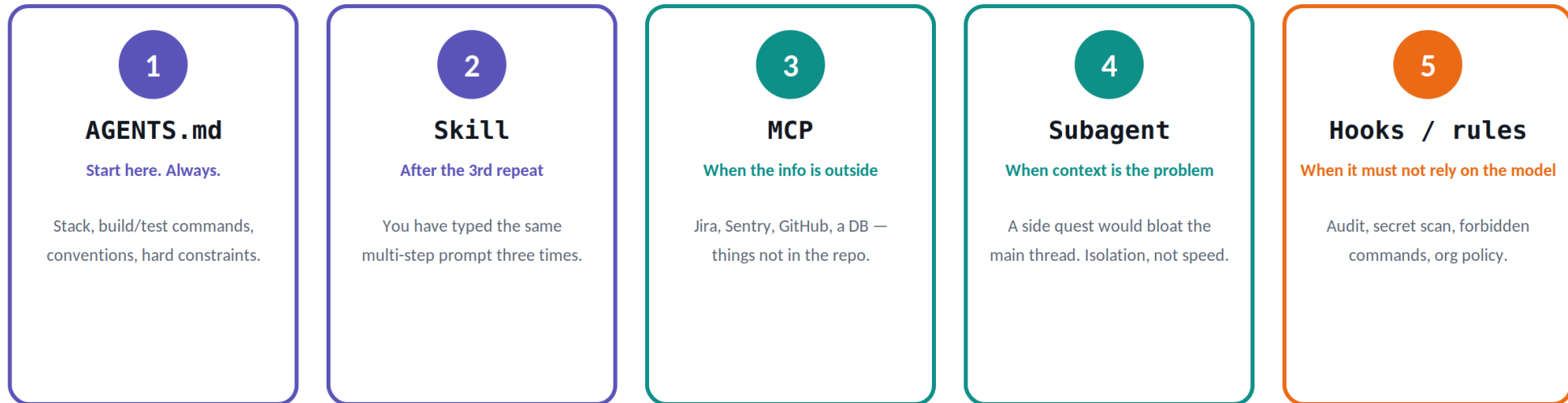
- One or two MCP servers, checked first, narrow tokens, enabled_tools cut down.
- A reviewer role with shell_tool = false — the one way a role can be narrowed.

Enforce

- Rules forbid what must never run; the sandbox is the actual boundary.
- Hooks observe and automate. They protect nothing on their own.

Move up a level only when the cheaper one runs out

Move up a level only when the cheaper one has actually run out



A good AGENTS.md plus two sharp skills beats a badly set up five-layer stack, every time.

Every layer you add is paid for on every model call, forever.

Start with AGENTS.md (/init creates one). Pull skills out after a week of real use. Add one MCP server for the outside info you need most. Only then subagents. Package as a plugin last, and only if you are sharing across repos.

Common mistakes — the ones that show up in real teams

Stuffing everything into AGENTS.md

Part 04 showed how that ends: the budget runs out from the root down and your most specific file is dropped, silently. Move detail into skills.

Using AGENTS.md for things that must happen

It is advice the model can ignore. "Never run migrations against prod" belongs in exec policy or the sandbox, not in a markdown file.

Registering servers "just in case"

Every one is a fixed cost on every model call, forever. Three good servers beat nine hopeful ones. Use `enabled = false`.

Delegating for speed

Subagents cost more tokens overall. Delegate to keep the parent's context clean, or do not delegate.

Several subagents writing at once

Agents editing at the same time produce real merge conflicts. Give each one its own worktree, or keep parallel work read-only.

A hook as the only safety net

When the script breaks, it lets everything through. If it must still hold then, it is not a hook's job.

Granting full permissions on day one

Test the workflow with a narrow sandbox first. Open it up on purpose, once you know what it actually needs.

Hiding the build and test commands

If AGENTS.md does not say how to run the tests, the agent will not run them. Agents do not add extra steps unless you tell them to.

Most of these never throw an error. The setup just quietly does less than you think it does.

Tips — keeping the context clean

Compact on your own terms

Part 05 covered the mechanics; the habit is the point. `/compact` at a checkpoint you choose, and remember that only `/clear` or a restart re-reads your files.

Delegate the expensive reading

"Find where X is implemented" and "check whether Y is covered by tests" are perfect subagent tasks: lots of reading, small answers.

Cut tools before cutting prompts

`enabled_tools` on one big MCP server usually saves more per call than any amount of prompt editing.

Watch the fixed block, not just the chat

`codex debug prompt-input` prints the fixed block before you have typed anything, and costs nothing. It prints the messages, not the tool schemas — so the real fixed cost is a little higher.

Small AGENTS.md, deep skills

The 32 KiB budget is spent from the repo root down, so a big root file deletes your most specific ones. Small is about staying correct, not saving money.

Treat the agent like a genuine new hire

FOR A NEW DEVELOPER

THE SAME THING, FOR THE AGENT

1 • Onboard

Reads the code and the docs, asks the team questions until the business makes sense.

Same — but you write down what it learns. One .md per feature, plus style, rules and tech debt. AGENTS.md says what to read and when.

2 • Give access

Accounts for the database, the repo, Jira.

MCP servers for those same systems. Read-only credentials wherever the job allows it.

3 • Train

You show them how your team actually does things.

The prompts your team keeps retyping become skills.

4 • Set limits

IT blocks some sites and stops you running random .exe files.

Exec policy for commands, hooks for the checks, the sandbox for the real barrier.

5 • One process

One way of working, so what the team produces is consistent.

The same flow written down in the repo, so every session produces the same shape of work.

One difference: a new hire remembers what you told them last week. The agent starts from zero every session — the file you wrote IS its memory.

Step 1 in detail — the part teams skip

What goes in the files

- One file per feature: what it does, why it exists, where the traps are.
- Style and conventions the team really enforces.
- Tech debt: what is wrong now, and the approach that replaces it.
- Libraries — free to use / ask first / never.

How to produce them

- Let the agent read the code and write the first draft. It is faster at that pass than anyone on the team.
- Then sit with it and ask questions. Your corrections are the valuable part.
- A person reviews every file before it is committed.

Keep them alive

- Every time you correct the agent, ask which file should have told it that.
- If none would have, add the line — or you teach the same thing again tomorrow.
- A wrong doc is worse than no doc: the agent believes every word of it.

```
# AGENTS.md — an index, not the content itself
## Read before you touch code
docs/agent/architecture.md      always
docs/agent/features/<area>.md    when the task is in that area
docs/agent/style.md             before writing any code
docs/agent/tech-debt.md         before "fixing" something that looks wrong
## Libraries
free to use   whatever is already in package.json
ask first    anything new, or anything that adds a build step
never        the list the team agreed on
```

Reviewing these files is the highest-leverage hour your team will spend — the agent trusts every line of them.

Codex CLI and Claude Code — the same ideas, different edges

CODEX CLI

CLAUDE CODE

Hooks

12 events. Windows needs its own `command_windows` line — a top cause of a hook never running.

33 events, finer-grained. shell: bash or powershell, chosen for you on Windows.

Hook failure

exit 0 / exit 2 / anything else. Exit 1 is reported, and the action still runs. Identical contract, identical behaviour. Both fail open.

Subagent role file

Changes model and instructions. `sandbox_mode`, `approval_policy` and `mcp_servers` parse fine and are ignored.

Really changes tools, `mcpServers`, `permissionMode`, `model`, `hooks` — and isolation: `worktree`.

Blocking a command

Exec policy: its own rule language, strictest wins, rules unit-tested at load, `execpolicy` check to dry-run.

`settings.json` deny / allow, e.g. `Bash(rm *)`. No rule tests, no dry-run. Permission modes instead.

Sandbox

Wraps the whole session: read-only / workspace-write / danger-full-access. Weak on native Windows.

Wraps Bash and its children only. Network via a domain-allowlist proxy. No native Windows — use WSL2.

Instruction file

`AGENTS.md` — one 32 KiB budget spent from the repo root down. The most specific file is dropped, no warning.

`CLAUDE.md` — 4 MiB, an oversize file is skipped. `@imports`. `.claude/rules/` with paths: loads only what matches.

Every idea in this deck applies to both. What differs is where each one lets you write the policy down.

Which one for which job

Reach for Codex when

The command policy has to be reviewed and tested.

Exec policy rules live in a file, carry their own unit tests, and you can dry-run a command against them before it runs.

Everything must be inside the sandbox.

It is a session-level setting, so a tool you forgot about is covered too.

Reach for Claude Code when

A delegated agent must really be narrower.

The role file applies tools, mcpServers and permissionMode instead of quietly ignoring them.

Several agents touch one repo at once.

isolation: worktree gives each its own checkout — no merge conflicts to untangle.

It barely matters when

You are writing or refactoring code in one repo.

Which is most of the work. Both read a markdown instruction file, both speak MCP, both use Seatbelt and seccomp, both hooks fail open.

The instruction file is doing the real work either way.

One thing that is true in both, and catches people in both

- A subagent cannot be stricter than a loose parent. Codex ignores sandbox_mode in a role file; Claude Code ignores permissionMode when the main session is in auto or accept-edits mode.
- So in either tool: set the session's permissions BEFORE you delegate. Do not expect the role file to tighten anything.

Pick per job, not per team. The habits carry over — write it down, verify live, never let a hook be the only barrier.

THE SENTENCE TO LEAVE WITH

"The model proposes. The harness decides."

MCP, skills, subagents, hooks, exec policy, the sandbox — every mechanism in these nine parts is a specific answer to how the harness decides, what it decides on, or how you steer that decision.

Guide it

and it may still ignore you

Extend it

and pay for that on every call

Enforce it

outside the model, or not at all

Questions